

MySQL Aggregate Functions and Window Functions

1. Aggregate Functions

Aggregate functions process multiple rows and return a summary result. They are commonly used when we need totals, averages, counts, highest values, or lowest values. When combined with GROUP BY, they can produce a separate summary for each group.

1. COUNT()

COUNT() counts rows or non-NULL values in a column. It is commonly used to find the total number of employees, customers, orders, or other records.

```
SELECT COUNT(*) AS Total_Employees
FROM Employees;
```

2. SUM()

SUM() adds numeric values in a column. It is commonly used to calculate total salary, total sales, total expenses, or other numeric totals.

```
SELECT SUM(Salary) AS Total_Salary
FROM Employees;
```

3. AVG()

AVG() calculates the arithmetic average of numeric values. It is commonly used for average salary, average age, average sales, and similar measurements.

```
SELECT AVG(Salary) AS Average_Salary
FROM Employees;
```

4. MAX()

MAX() returns the highest value in a column. It can be used to find the highest salary, maximum sales, oldest age, or latest value.

```
SELECT MAX(Salary) AS Highest_Salary
FROM Employees;
```

5. MIN()

MIN() returns the lowest value in a column. It can be used to find the lowest salary, minimum sales, youngest age, or earliest value.

```
SELECT MIN(Salary) AS Lowest_Salary
FROM Employees;
```

Aggregate Functions with GROUP BY

GROUP BY divides records into groups. The aggregate function then calculates one summary value for each group.

```
SELECT Department,
COUNT(*) AS Total_Employees,
SUM(Salary) AS Total_Salary,
AVG(Salary) AS Average_Salary,
MAX(Salary) AS Highest_Salary,
MIN(Salary) AS Lowest_Salary
FROM Employees
GROUP BY Department;
```

2. Window Functions

A window function performs a calculation across a set of related rows while normally keeping every original row in the result. Window functions use the OVER() clause. PARTITION BY divides rows into groups, and ORDER BY controls the order used for the calculation.

1. ROW_NUMBER()

ROW_NUMBER() assigns a unique sequential number to each row according to the specified order. With PARTITION BY, the numbering starts again inside each group.

```
SELECT EmployeeID, EmployeeName, Salary,
ROW_NUMBER() OVER (ORDER BY Salary DESC) AS Row_Num
FROM Employees;
```

2. RANK()

RANK() gives the same rank to tied values. After a tie, the next rank is skipped, so gaps can occur in the ranking.

```
SELECT EmployeeID, EmployeeName, Salary,
RANK() OVER (ORDER BY Salary DESC) AS Salary_Rank
FROM Employees;
```

3. DENSE_RANK()

DENSE_RANK() gives the same rank to tied values but does not skip the next rank. Therefore, the ranking has no gaps.

```
SELECT EmployeeID, EmployeeName, Salary,
DENSE_RANK() OVER (ORDER BY Salary DESC) AS Salary_Rank
FROM Employees;
```

4. NTILE()

NTILE(n) divides ordered rows into a specified number of groups. It is useful for creating quartiles, performance groups, or customer segments.

```
SELECT EmployeeID, EmployeeName, Salary,
NTILE(4) OVER (ORDER BY Salary DESC) AS Salary_Group
FROM Employees;
```

5. LAG()

LAG() retrieves a value from a previous row. It is useful when comparing the current row with an earlier row.

```
SELECT EmployeeID, EmployeeName, Salary,
LAG(Salary) OVER (ORDER BY EmployeeID) AS Previous_Salary
FROM Employees;
```

6. LEAD()

LEAD() retrieves a value from a following row. It is useful when comparing the current row with a later row.

```
SELECT EmployeeID, EmployeeName, Salary,
LEAD(Salary) OVER (ORDER BY EmployeeID) AS Next_Salary
FROM Employees;
```

7. FIRST_VALUE()

FIRST_VALUE() returns the first value in the selected window according to its ordering. It is useful for comparing each row with a first or top value.

```
SELECT EmployeeID, EmployeeName, Salary,
FIRST_VALUE(Salary) OVER (ORDER BY Salary DESC) AS Highest_Salary
FROM Employees;
```

8. LAST_VALUE()

LAST_VALUE() returns the last value in the selected window. A complete window frame is often specified so that the final row of the full window is used.

```
SELECT EmployeeID, EmployeeName, Salary,
LAST_VALUE(Salary) OVER (
ORDER BY Salary
ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING
) AS Lowest_Salary
FROM Employees;
```

9. SUM() OVER()

SUM() becomes a window calculation when combined with OVER(). It can calculate a running total or a total within each partition while retaining every row.

```
SELECT EmployeeID, EmployeeName, Salary,
SUM(Salary) OVER (ORDER BY EmployeeID) AS Running_Total
FROM Employees;
```

10. AVG() OVER()

AVG() becomes a window calculation when combined with OVER(). With PARTITION BY, it calculates an average separately for each group while retaining every employee row.

```
SELECT EmployeeID, Department, Salary,
AVG(Salary) OVER (PARTITION BY Department) AS Avg_Salary
FROM Employees;
```

11. COUNT() OVER()

COUNT() becomes a window calculation when combined with OVER(). It can count rows inside each partition while keeping every original row.

```
SELECT EmployeeID, EmployeeName, Department,
COUNT(*) OVER (PARTITION BY Department) AS Department_Count
FROM Employees;
```

12. MAX() OVER()

MAX() becomes a window calculation when combined with OVER(). It can display the highest value across the complete window or within a partition.

```
SELECT EmployeeID, EmployeeName, Salary,
MAX(Salary) OVER () AS Highest_Salary
FROM Employees;
```

13. MIN() OVER()

MIN() becomes a window calculation when combined with OVER(). It can display the lowest value across the complete window or within a partition.

```
SELECT EmployeeID, EmployeeName, Salary,
MIN(Salary) OVER () AS Lowest_Salary
FROM Employees;
```

3. Aggregate Function vs Window Function

The most important difference is what happens to the original rows. Aggregate functions summarize data and, when used with GROUP BY, produce fewer rows. Window functions calculate across related rows but normally keep every individual row in the result.

Feature	Aggregate Function	Window Function
Main purpose	Summarizes multiple rows.	Calculates across related rows while retaining row-level details.
Rows returned	Can reduce rows to one result or one result per GROUP BY group.	Normally keeps all original rows.
Typical syntax	AVG(Salary), SUM(Salary), COUNT(*)	AVG(Salary) OVER(...), SUM(Salary) OVER(...)
GROUP BY	Commonly used with GROUP BY.	Uses OVER(); PARTITION BY can divide rows into groups.
Ranking	Not designed for ranking rows.	ROW_NUMBER(), RANK(), DENSE_RANK(), and NTILE() provide ranking/grouping.
Previous/next row	Does not directly access previous or next rows.	LAG() and LEAD() access previous and next rows.
Typical question	What is the average salary of each department?	Show every employee and the average salary of their department.

Example 1: Aggregate AVG()

If we only need the average salary for each department, use AVG() with GROUP BY. The result contains one row per department.

```
SELECT Department,
AVG(Salary) AS Average_Salary
FROM Employees
GROUP BY Department;
```

Example 2: Window AVG()

If we need every employee and also want the average salary of that employee's department, use AVG() as a window function. The department average appears beside each employee.

```
SELECT EmployeeName,
Department,
Salary,
AVG(Salary) OVER (PARTITION BY Department) AS Average_Salary
FROM Employees;
```

4. Practical Difference in Simple Terms

Aggregate: "Give me the average salary of each department." The result is a department-level summary.

```
SELECT Department, AVG(Salary) AS Average_Salary
FROM Employees
GROUP BY Department;
```

Window: "Show every employee and also show the average salary of that employee's department." The result keeps all employees.

```
SELECT EmployeeName, Department, Salary,
AVG(Salary) OVER (PARTITION BY Department) AS Average_Salary
FROM Employees;
```

5. OVERALL

- Aggregate functions are mainly used for summaries: COUNT(), SUM(), AVG(), MAX(), and MIN().
- GROUP BY is used when the summary should be produced separately for each group.
- Window functions use OVER() and normally keep the original rows in the output.
- PARTITION BY divides rows into groups for a window calculation without collapsing those rows.
- ORDER BY inside OVER() controls the order for ranking, previous/next-row calculations, and running calculations.
- ROW_NUMBER(), RANK(), DENSE_RANK(), and NTILE() are mainly used for ranking or grouping rows.
- LAG() accesses a previous row and LEAD() accesses a following row.
- SUM() OVER(), AVG() OVER(), COUNT() OVER(), MAX() OVER(), and MIN() OVER() provide aggregate-style calculations while retaining row-level details.